AI Basketball Practice Coach

Implementation status report

22 October 2026 | Innovative-Design-Project/pose-estimation, main @ 765c86e (pushed to GitHub)

1. Summary

Every v1 feature in the project plan now exists in code: live coaching with voice, shot, make/miss, dribble and layup detection, 12 form metrics, hand landmarks, a session database, offline analysis and an analytics dashboard. It has been tested with 16 automated tests, a synthetic shooter, two recorded webcam sessions and a basketball broadcast clip.

It has not been tested on real practice footage yet, because the team could not record any. Shot detection thresholds, make/miss accuracy and the coaching target ranges are therefore starting values, not measured ones. That footage is now the main blocker for the rest of the project.

Three things are needed to move forward

1. **Practice footage:** 3-5 short shooting sessions recorded with live.py from the agreed camera position, with makes and misses counted by hand.
2. **Security follow-up:** a VS Code auto-run malware payload was found and removed from this repository (see section 6). Every teammate should pull, scan their machine and change credentials.
3. **API keys (optional):** ELEVENLABS_API_KEY for the real voice and a Claude Console ANTHROPIC_API_KEY for natural cue wording and summaries. Without them the app falls back to macOS *say* and rule-based wording.

Roadmap status

Phase (from the plan)	Status	Notes
0. Foundations	**Verified**	Live camera loop, recording, model default fixed; ~28 fps on the M4 webcam
1. Perception	**Code done**	Player lock, smoothing, ball detector trained, rim calibration, hand landmarks; pose accuracy on practice footage not measured
2. Shooting	**Needs footage**	Shot / make-miss / form metrics work on synthetic shots; thresholds untuned
3. Live coaching	**Needs key**	Rules, cue choice and voice pipeline work; Claude and ElevenLabs untested without keys
4. Analytics	**Verified**	Session store, analyze.py and dashboard checked end to end with demo data
5. Dribbling & layups	**Needs footage**	Dribble counting exact on synthetic data; layup steps approximate
6. Validation	**Blocked**	Needs practice sessions and a coach to review cues

2. What has been implemented

Component	Files	Status	What it does / evidence
Live mode	live.py	**Verified**	Webcam/phone/file input, threaded capture, overlay, recording (raw.mp4, timestamps, keypoints), rim box at start. 95% of frames analysed at 30 fps with ball detection.
Player lock + smoothing	coach/tracking.py	**Verified**	Locks the practising player, ignores bystanders; One-Euro filter removes keypoint jitter. Unit-tested.
Ball detector	sports_pipeline/train_ball.py, coach/ball.py	**Verified**	YOLO11n trained 15 epochs: precision 0.89, recall 0.81, mAP50 0.87. Tracks start only near the player.
Rim calibration	coach/hoop.py	**Code done**	Drag a box once per session; --hoop last reuses it. Needs a real court to try.
Hand landmarks	coach/hands.py	**Code done**	MediaPipe 21 points per hand on wrist crops, only around shots (~11 ms). Wrist-snap angle; experimental.
Shot detection	coach/events.py	**Needs footage**	Pose-triggered (works without the ball), guide hand excluded, release refined by the ball. 24/24 synthetic shots.
Make / miss	coach/events.py	**Needs footage**	Ball crossing the rim plane inside the rim box. Correct on synthetic makes and misses.
Dribbles, layups	coach/events.py	**Needs footage**	Bounce count, pace, height, hand; layup steps and take-off foot (approximate).
Form metrics	coach/metrics.py	**Needs footage**	12 metrics incl. elbow, knee bend, release height/timing, arc (fitted parabola), follow-through, balance, wrist snap.
Coaching cues	coach/coaching.py	**Code done**	Target ranges, one cue per shot, praise, no repeating the same cue 3 times; dribble and layup feedback.
Claude wording + summary	coach/llm.py	**Needs key**	Rephrases cues and writes session reports; falls back to rule text without a key (verified).
ElevenLabs voice	coach/voice.py	**Needs key**	Streams speech in the background with a cache; falls back to macOS say (verified).
Session store	coach/store.py	**Verified**	SQLite: sessions, shots with metrics and cues, dribbles.
Offline analysis	analyze.py	**Verified**	Full pipeline on a session folder or any video with the more accurate pose model.
Analytics dashboard	dashboard.py	**Verified**	KPIs, shot timeline, rolling FG%, shot key frames, trends vs targets, consistency, dribbling, session comparison, CSV export, Claude summary button.
Pose fine-tune / eval tools	sports_pipeline/*	**Built, not run**	Clip-level split, CVAT round trip, evaluate.py; waiting for hand-labelled test frames.
Tests	tests/	**Verified**	16 tests with a synthetic shooter; all pass.

3. Measured results

Check	Result	Data
Automated tests	16 / 16 pass	Synthetic shooter at 15, 25 and 60 fps
Synthetic sessions	24 / 24 shots detected; dribbles 12/12 and 20/20	Generated shots with varied arcs, makes and misses
False shots on non-shooting video	0	Two webcam sessions (arm raised, fist)
Live speed, pose + ball	95% of frames (~28 fps)	30 fps replay, Apple M4 GPU
Live speed, + hand landmarks	71% (~21 fps)	Arm raised most of the clip, so hands ran often
Ball detector (validation)	P 0.89, R 0.81, mAP50 0.87, mAP50-95 0.61	501 held-out frames
Ball on broadcast clip	Detected in 18% of frames at 640 px, 82% at 1280 px	Ball ~10 px wide; crowd causes false positives
Engine overhead	~1 ms/frame; ~11 ms with hands	Excluding the pose and ball models

The broadcast clip is much harder than practice footage (tiny ball, red crowd, many players), so it cannot tell us real-world tracking accuracy. Only practice footage can.

4. What needs to be done

Owner: **Team** = needs people or hardware; **Code** = development work. Priority: P0 blocks progress, P1 needed for v1, P2 later.

Task	Owner	Pri.	Blocked by / notes
Record 3-5 practice sessions (side/45 degrees, 4-6 m, body + rim in frame, 60 fps); count shots and makes by hand	Team	P0	Access to a court
Every teammate: pull the cleaned repository, scan their machine, change credentials (see section 6)	Team	P0	-
Share the trained ball model (best.pt, 5.5 MB) with the team, or retrain (~2.5 h)	Team	P0	Model files are git-ignored
Measure shot detection and make/miss accuracy against the hand counts	Code	P0	Practice footage
Tune shot trigger thresholds and form target ranges on real shots	Code + coach	P0	Practice footage
Fine-tune the ball detector on our footage if it misses balls	Code	P1	Practice footage + ~200 labelled frames
Hand-label the 251 test frames in CVAT and run the pose baseline (evaluate.py)	Team	P1	Sri's CVAT skeleton setup can be reused
Check whether wrist-snap (hand landmarks) is stable enough to coach on	Code	P1	Close-up release frames
Add ELEVENLABS_API_KEY and test live voice latency (target <2 s)	Team	P1	ElevenLabs account (free tier OK)
Add a Claude Console ANTHROPIC_API_KEY (COACH_LLM_MODEL=claude-haiku-5-5 ~ $0.03 per session)	Team	P1	A Claude Pro subscription does not include API access
Coach review of cues and target ranges	Team	P1	A coach or experienced player
Court-distance zones (shot chart by distance)	Code	P2	Court calibration
Remove hockey paths and other leftovers; delete stray nul file	Code	P2	-

5. Known limitations

- All angles are 2D from one camera, so they are only comparable between sessions filmed from the same position.
- Thresholds and target ranges are textbook starting values; none are tuned on our players yet.
- Make/miss needs the rim box and a tracked ball; without the ball, shots are found but the result is 'unknown'.
- Layup step counting and the 'head up' idea for dribbling are approximate in 2D.
- Hand landmarks are unreliable when the hand is small in the frame (about 25-35 px at courtside distance).
- Claude and ElevenLabs need internet; offline the app still coaches with rule wording and the system voice.
- The ball detector was trained on game footage; practice footage should be easier but is unmeasured.

6. Repository status

Commit	Change
765c86e	Merge of Sri's CVAT test dataset and README content (without the payload)
7132027	Security: removed VS Code auto-run malware and fake font payloads
582977d	Ball tracks start near the player; --ball-imgsz; resumable ball training
03e93b6	Live coaching, analyze.py, dashboard.py, README
9aacd56	Coaching cues, Claude phrasing, ElevenLabs voice, store, engine, tests
2616676	Tracking, ball/rim/hands, event detection, form metrics
f888127	Ball detector training script
6cad7fc	Dependencies (OpenCV/numpy pins, mediapipe, streamlit, anthropic)
384deae	Live mode: pose off the main thread (59% to 87% of frames)
d39ecfa	Live camera mode with session recording; plan report
66f6d98, f76fda9	Basketball-only assets; clip-level split, CVAT flow, evaluation script

All of the above is on GitHub, merged with Sri Saidhakshini's CVAT test dataset (5 labelled images) and README content in 765c86e.

Security incident

While merging, we found malware in the repository: **.vscode/tasks.json** ran a hidden task on every folder open that executed fake "font" files (obfuscated JavaScript) under outputs/eval_runs/rtdetr-l/public/fonts, and **.vscode/settings.json** enabled automatic tasks and hid the terminal. It arrived in commits made from team members' machines (29 Aug, 17 Sep, 22 Sep) and very likely ran on at least one laptop. It was removed in 7132027 and the GitHub copy now scans clean. The same payload was also cleaned from another of Gowreesh's repositories (DnG).

- Pull the cleaned main; never open an old checkout of this repository in VS Code.
- Set **task.allowAutomaticTasks** to **off** in VS Code user settings.
- Scan your machine and change GitHub, API-key and saved browser passwords from a clean device.
- Review new commits for changes to .vscode/, font files or JS config files before merging.
- Old commits still contain the payload; rewriting history is a team decision.

How to run

Purpose	Command
Install	pip install -r requirements.txt
Live coaching	python live.py (drag a box around the rim, press Enter; q to stop)
Re-analyse a session	python analyze.py sessions/<session-folder>
Analytics dashboard	streamlit run dashboard.py
Train / resume ball detector	python sports_pipeline/train_ball.py --device mps (or --resume)
Tests	pytest tests